```
    {                                                          ❷
        Value = 1,                                             ❷
        ExchangeRate = -2,                                     ❷
        DecimalPlaces = 2,                                     ❷
    };                                                         ❷

    controller.ModelState.AddModelError(                       ❸
        nameof(model.ExchangeRate),                            ❸
        "Exchange rate must be greater than zero"              ❸
    );                                                         ❸

    ActionResult<decimal> result = controller.Convert(model);  ❹

    Assert.IsType<BadRequestObjectResult>(result.Result);      ❺
}
```

❶ Creates an instance of the Controller to test

❷ Creates an invalid binding model by using a negative ExchangeRate

❸ Manually adds a model error to the Controller's ModelState. This sets ModelState.IsValid to false.

❹ Invokes the action method, passing in the binding models

❺ Verifies that the action method returned a BadRequestObjectResult

NOTE In listing 36.6, I passed in an invalid model, but I could just as easily have passed in a valid model or even `null`; the controller doesn't use the binding model if the `ModelState` isn't valid, so the test would still pass. But if you're writing unit tests like this one, I recommend trying to keep your model consistent with your `ModelState`; otherwise, your unit tests won't be testing a situation that occurs in practice.

I tend to shy away from unit testing API controllers directly in this way. As you've seen with model binding, the controllers are somewhat dependent on earlier stages of the MVC framework, which you often need to emulate. Similarly, if your controllers access the `HttpContext` (available on the `ControllerBase` base classes), you may need to perform additional setup.